Journal of Systems Architecture 107 (2020) 101774

Contents lists available at ScienceDirect

Journal of Systems Architecture

journal homepage: www.elsevier.com/locate/sysarc

Build real-time communication for hybrid dual-OS system

Pan Dong [a,b], Zhe Jiang [a,*], Alan Burns [a], Yan Ding [b], Jun Ma [b]

[a] Computer Science Department, University of York, YO10 5GH, UK
[b] College of Computer Science and Technology, National University of Defense Technology, Changsha, Hunan Province, PR China

A B S T R A C T

In automotive/avionic/robot systems, consolidating multiple domains on a powerful SoC is an important trend. From the perspectives of safety and security, each of these domains must execute independently with a self-governed Operating System (OS). Such systems are termed as hybrid multi-OS systems. Generally, in a commonly used hybrid dual-OS system, a real-time OS (RTOS) and a general-purpose OS (GPOS) are integrated to satisfy different requirements. Inter-OS communication is the key to such systems. Specifically, services are often requested by tasks in RTOS from GPOS, but it is challenging to satisfy the requirements on time efficiency and predictability in RTOS, due to the uncertainty in communication maintenance and the time-sharing policy of the GPOS. Current existing approaches can not provide predictable time guarantees to the circumstances.

This paper aims to build a time predictable and secure RPC mechanism (i.e., the primary and critical communication unit in a hybrid multi-OS system). On the foundation of a detailed analysis of the time delays in each phase of a RPC from RTOS to GPOS, we point out the drawbacks in conventional RPC mechanisms and propose an effective model and some methods to guarantee the predictability in all the phases of RPC.

Furthermore, a real-time RPC scheme (termed RTRG-RPC) is proposed and applied to our ready-built TrustZone-based consolidation system (i.e., TZDKS). RTRG-RPC achieves accurate time control through three mechanisms: SGI (Software Generate Interrupt) message transforming, interrupt handler RPC servicing, and priority-swapping. Evaluations show that RTRG-RPC can achieve real-time predictability and managed priority inversion. Moreover, the design of RTRG-RPC also provides lightweight and essential protection on the security, without violation of the efficiency.

1. Introduction

Consolidating multiple domains on single powerful SoC has become a popular trend in automotive/avionic/robot systems [1], which meets stringent requirements relating to cost, space, weight, heat generation, power consumption. Take the automotive system as an example, three sub-systems named ECU (Electronic Control Units) cluster, ADAS(Advanced Driver-Assistance System), and IVIS(In-Vehicle Information System) are recommended for a vehicle in the AUTOSAR specification with different capability requirements (see Table 1). The ECU control sub-system claims high demand in safety, security and real-time, but low demand in data processing and versatile I/O. While the IVIS sub-system claims almost opposite demands. Therefore, the final platform is a hybrid system with different characteristics [1]. These systems are named as hybrid OS in some researches [2], and we further named them hybrid multi-OS systems in this paper. The simplest form of a hybrid Multi-OS system is the composition of a real-time OS (RTOS) and a general-purpose OS (GPOS), i.e., a hybrid dual-OS system. Communication is the foundation to build efficient functions in such hybrid systems. For instance, with the assistance of communication, the ECU cluster in the RTOS can acquire abundant functionalities through the inter-operation with the IVIS in the GPOS (e.g., fault logs, cloud-side AI decisions, etc.).

In the hybrid multi-OS system, communication raises more stringent requirements. Firstly, it should be efficient and predictable [3–6]. As the most crucial subsystem, the RTOS part requires that all its mechanisms, including communications, are predictable. Secondly, it must be safe and secure. A hybrid system usually serves as or contains a safety-critical system, so the communication should not do harm to validity, reliability, and integrity. It should also prevent interference from different domains/OSs that may harm other digital assets, especially the high criticality subsystems. Unfortunately, these two goals are beset by difficulties and contradictions. As one of the participants, the GPOS is usually not real-time designed, so the communication may be blocked and preempted by less-critical tasks. Actually, the deeper problem is that the GPOS can not provide necessary support for the time analysis of the communication. In the prior work[7–11] about the hybrid systems, many optimization measures (e.g., zero-copy, hardware assistant) were proposed to increase the efficiency of the communication, but real-time supporting has not been included in the design for long time. Such a dilemma is caused by lacking appropriate communication model for the system with high complexity. The GPOS always supports internet downloading applications, which induce serious risk about safety and security – i.e., perhaps threat the RTOS by communication. When the common mechanisms [12–14], such as audit or signature, are exerted on the communication, the efficiency and predictability will be obviously weakened. As far as we know, there is very few literature addressing such problems, especially the time predictable requirements in commu-

* Corresponding author.
E-mail address: zhe.jiang@york.ac.uk (Z. Jiang).

https://doi.org/10.1016/j.sysarc.2020.101774
Received 30 July 2019; Received in revised form 11 February 2020; Accepted 17 March 2020
Available online 28 April 2020
1383-7621/Crown Copyright © 2020 Published by Elsevier B.V. All rights reserved.

Table 1
Capability requirements difference among ECU, ADAS and IVIS.

	Safety/ Security	Real-time	Graphics/Data processing	Versatile I/O
ECU	High	High	Low	Low
ADAS	High	Low	High	Low
IVIS	Low	Low	High	High

nication, which may be a serious obstacle to promote the development of the consolidation system.

In this paper, we propose a real-time RPC as a fundamental mechanism [10] of the communication in a consolidation system. The contributions of the paper are summarized as:

- We set up a procedure model for RPC in a hybrid multi-OS system, which is suitable for the time analysis. Based on the model, we also highlight the efficiency and the security problems existed in the current RPC protocols, and point out the key links corresponding to such problems.
- We propose a RTRG-RPC (Real-Time RTOS-GPOS Remote Procedure Call) protocol on TZDKS [15] (i.e., a TrustZone-based hybrid multi-OS system). The time predictability of RTRG-RPC is achieved via three mechanisms: SGI (Software Generate Interrupt) message transforming, interrupt handler RPC serving, and priority-swapping.
- We also propose the security design for RTRG-RPC, which obtains reliability and resists malicious attacks and DoS attacks. Furthermore, the proposed design does not violate the efficiency and the predictability of RPC.
- We conducted a performance evaluation on the TZDKS platform. The results show that RTRG-RPC can achieve real-time predictability and reduce priority inversion. The results also prove that our security design does not violate the efficiency object of the RPC protocol.

In fact, the design of RTRG-RPC is generic, which can be directly applied to other hybrid multi-OS systems.

The paper is organized as follows: Section 2 introduces the related work of hybrid multi-OS system and inter-OS communication. Section 3 gives a brief description of TZDKS. Section 4 presents our model for RG-RPC and summary the problems concerned in such communication. Section 5 describes the design philosophy and the implementation of RTRG-RPC. Section 6 evaluates the performance of RTRG-RPC, with conclusions offered in Section 7.

2. Background and related work

Hybrid multi-OS system significantly contributes to the modern industrial manufacturing requirements, i.e., cost, space, and power consumption. Therefore, it represents the development direction of many comprehensive automatic systems such as avionics and automotive computer systems. The up-to-date avionics systems specification - ARINC 653 [16] requires integrating flight control systems, environment control systems, and amusement systems into a virtualized platform on modern aircraft. Meanwhile, AUTOSAR 4.0+ (Automotive Open System ARchitecture) proposes ECU consolidation in a modern vehicle based on virtualization [1,17,18]. Even more, the consolidation of ECU, ADAS, and IVIS subsystems will be the final trend [1]. One of the popular candidates for consolidation is isolating the sub-system into the different run-time environments (e.g., virtualization technology — running each sub-system in independent virtual machines). However, using virtualization technology significantly conflicts with the requirements on resource efficiency and predictability, due to the introduction of complicated resource management and complex access paths. The other method is building the sub-systems in the isolated environments provided as the extension by the SoC hardware, such as ARM TrustZone [19]. For instance, TZDKS [15] and LTZVisor[20] proposed multi-OS architectures upon TrustZone technologies, which achieve better system performance compared to normal virtualization approaches.

Table 2
Latency Statistic for RSA and ECC Signatures.

Key Size(bit)	Data Size(bit)	Signature Time(ns)	Verify Time(ns)
RSA-1024	128	31,728,412	638,329
ECC-224	63	45,102,993	69,212,984
RSA-3072	384	183,469,702	3,366,627
ECC-256	71	55,389,519	84,698,999

2.1. Efficient and secure communication related to consolidation system

Current inter-OS communication mechanisms are mostly designed for virtualization systems. The default method is to route messages via the standard network interface. This offers the highest amount of isolation, yet provides the lowest performance. Many improvements simplify the under protocol stack and use shared-memory to increase the performance [7]. Examples include XenLoop, MemPipe [8], etc. Another effort is to design more straightforward RPC with direct hardware assistants. XENRPC [9] is a product of such an idea. In some dual-OS systems, the characteristics of a special platform are leveraged to build efficient communication. As shown in the design of SafeG [10], efficient dual-OS communication protocols are proposed by using SGI as the event path and using shared memory as the data path. All the above efforts are focused on optimization for high performance, and have no consideration for the predictability.

In other OS research fields, many new hardware associated capabilities are utilized or proposed to strengthen the communication performance. CrossOver [21] uses virtualization support available in commodity processors (specifically the VMFUNC instruction) to support efficient cross-world calls. dIPC [22] allows that threads access each other by crossing process boundaries with the help of new CODOM architecture. XPC [23] enables direct switch between IPC caller and callee without trapping into the kernel by adding hardware-assisted OS primitives to the CPU core. These schemes lack practicability in the frequently-used hardware platforms, so it is challenging to apply them in the current hybrid multi-OS system.

On the hybrid system where components at different security levels coexist, sub-systems are protected from others by using isolation. Therefore, the communication design should maximally preserve the isolation. As far as we know, two kinds of main protection were adopted for the RPC protection on the similar system architectures. The first one is based on cryptography, and Secure-RPC/Secure-XML-RPC[12] belong to this category. As another kind of protection method, Arash al [13] exerts Mandatory Access Control on the RPC protocols. These protections will depend on the complex and heavy load software layer (For example, one time of digital signature will consume more than 30ms on a 2.2GHz x86 processor, as shown in Table 2), and will make it difficult to ensure the predictability of RPC process.

3. TZDKS architecture

TrustZone [24] is a hardware-based security extension technology incorporated into ARM processors, which provides two isolated environments on a single physical platform: the normal world and the secure world. Access permissions are strictly under the control of the secure world, which forbids access to the protected resources from the normal world.

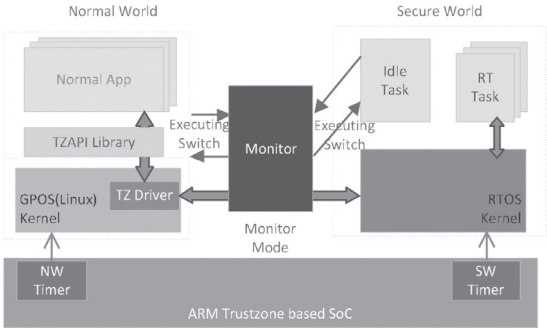


Fig. 1. TZDKS architecture.

TZDKS [15] was proposed to consolidate GPOS and RTOS based on TrustZone isolation efficiently. Its design idea comes from our investigation of dual-kernel [15] system and virtualization. Both methods can consolidate GPOS and RTOS. In the comparison, dual-kernel system achieves better performance in term of real-time latency and has less overhead without the introduction of complicated resource management and complex paths of instructions, though the virtualized system is better than a dual-kernel system in aspects of complexity and isolation. By utilizing TrustZone technology, TZDKS combines strong points of dual-kernel and virtualization. Firstly, TZDKS adopts a dual-kernel structure to support applications of different time-sensitivities. On the one side, all tasks in RTOS can be controlled in a time-deterministic way, because (i) the RTOS kernel is independently driven by a high-resolution timer, (ii)RTOS support pre-empted fixed priority scheduling and (iii) GPOS executes integrally as a scheduling unit in RTOS. On the other side, idle time in RTOS will be fully utilized by GPOS to execute none-real-time applications, and the throughput or interactive strategies will still work in GPOS. Secondly, TZDKS utilizes TrustZone technology to get performance trade-off. TrustZone extension provides isolation while ensuring efficient access to resources.

As shown in Fig 1, there are two software stacks located in the two worlds of TrustZone-enabled environment on TZDKS. The secure world stack is composed by the monitor module, RTOS and real-time tasks/services, and provides a real-time environment for the development of applications which need to guarantee specific deadlines. While the normal world stack is composed of GPOS and applications, and provides a rich environment for running user-machine interfaces as well as internet-based applications and services.

4. RG-RPC model and problem summary

There are two different types of RPC: (1)RG-RPC defines the RPC from a RTOS task to an entity in the GPOS. (2)GR-RPC defines the RPC from a task from GPOS to RTOS. In most of cases the RG-RPC require higher demand for time predictability, so we only concern RG-RPC in this paper, without loss of generality.

In order to investigate RG-RPC in detail, we try to break it into contiguous sub-processes, and every sub-process is indispensable and consists of a strongly ordered actions assemble. Note that strong order indicates some actions in order are bound to exist in all succeed RPC processes. Such a process of RG-RPC is shown in Fig 2. This model fits well for RG-RPC, because each component runs exclusively on a uni-processor (We mainly consider the process on a uni-processor platform, because it can be adapted to a multi-core system or a distributed environment with simple extension). As the illustration, When a task τ_i triggers a RPC request, it will sleep until a RPC answer received, or timeout. The RG-RPG process is constituted by five essential parts (the sequence may be different), which are:

1. Issuing a RPC request. τ_i triggers a kernel function to start a RG-RPC, then sleeps.

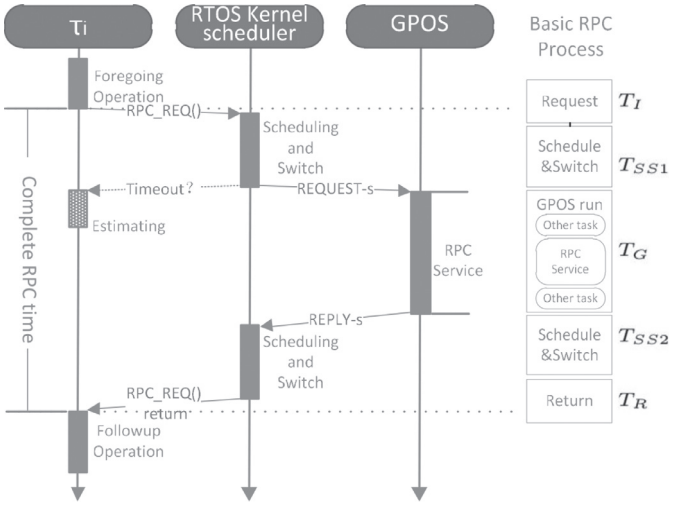


Fig. 2. Basic Process of RG-RPC on Uni-processor.

2. Scheduling and context switch of RTOS kernel. GPOS gets a chance to execute after this. In this block, tasks in RTOS may be switched on or off.
3. GPOS executing. GPOS has its own scheduling mechanism, so the RPC service will be switched on by the kernel at some time-point.
4. Another round of scheduling and context switch by RTOS kernel. τ_i should be awoken and feed into the ready queue, then wait for being switched on.
5. Return of RPC. τ_i is switched on, and the RPC call returns, then follow-up operations will be executed.

We define these five parts as T_I, T_{SS1}, T_G, T_{SS2}, T_R respectively in a sequence block as shown in the right of Fig 2.

As the communication may break the isolation between the OSs, two classes of the problems should be taken into concern and resolved in its design: efficiency and security.

4.1. Efficiency problems

The communication should meet the timing requirement so that a request can be served and returned in time. So we plan to build the full RPC mechanism at the bottom layer of the software without dependency on any mature communication infrastructures. Even so, due to the following issues, it is still challenging to satisfy the real-time constraints:

1. The first barrier comes from the system priority of GPOS, which is lower than all working tasks in the RTOS. Thus, the GPOS can not get the CPU controlling until there is no ready RT task (decided by T_{SS1} in Fig 2), and that means the RPC service can hardly get a chance to run.
2. As the second barrier, GPOS itself is a non-real-time system, so the RPC service time is not deterministic (T_G part in Fig 2) even if the GPOS is running. Take the scheduling of Linux as an example. In Fig 3 quoted from paper [25], we can see that the latency of schedules is too long and unpredictable.
3. The third barrier lies in the RPC scheduling. Due to the lacking of a preemption mechanism designed for RPC, a higher priority RPC may be blocked by a lower priority one(probably in T_{SS1} or T_{SS2} in Fig 2).

4.2. Security problems

Actually, this property contains both safety and security, so we have three problems to be resolved.

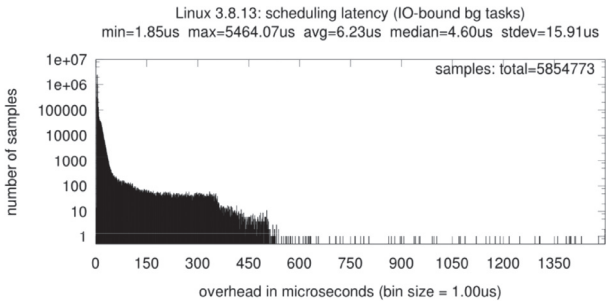


Fig. 3. Scheduling Latency Testing in Linux.

1. For safety, each communication must not lead to or propagate hazards and faults. In the hybrid multi-OS system such as TZDKS, the RPC request may lead to the suspend of the related task, or OS switch, so sufficient mechanisms must be prepared to cope with various exceptions such as time out and state restoring. If one OS gets into a fault, it may trigger incorrect messages or requests, and such errors should not lead to faults in the other OSs.
2. For security, a malicious task can not threat other OSs or get private information through a deliberate message. In TZDKS, the RTOS environment is protected by the TrustZone extension, so the codes in other OSs are forbidden to access any resources in RTOS. However, the communication may break such protection and provide a window to GPOS, by which some male-wares can even intrude RTOS. Some examples [26,27]show how to leverage the communication and vulnerabilities to attack the TrustZone protection. Besides, many communication requests may trigger DoS (Denial of Service) of related OSs.
3. As mentioned in Section 2, all protection methods will induce overheads. How can we get the protection design without any violation to the time predictability?

5. Design of RTRG-RPC

5.1. Design philosophy

Based on the analysis in Fig 2, the response time of RG-RPC is computed as:

$$T_{RG-RPC} = T_I + T_{SS1} + T_G + T_{SS2} + T_R \tag{1}$$

We can deem that T_I and T_R are the duration of the function call and return, respectively. Each duration is fixed and trivial. Therefore, we mainly consider the other three parts.

- T_{SS1}. For the predictability of T_{SS1}, we should let the GPOS be switched on the CPU as soon as possible. Meanwhile, tasks with higher priority than τ_i can also pre-empt in this process. T_{SS1} can be calculated as

$$T_{SS1} = T_{rsc} + T_{rrc} + T_{sched} + T_{msr} + T_{mrg} + T_P \tag{2}$$

  Here T_{rsc} and T_{rrc} respectively link to the duration to save and restore the context of task by RTOS, while T_{msr} and T_{mrg} respectively denote the duration to save the context of RTOS and the duration of restoring the context of GPOS by the monitor, and T_P is the interference which is brought by other tasks in a special period. In TZDKS, GPOS is scheduled under the idle-scheduling policy [15]. So GPOS has no chance to execute until there is no ready task of RTOS. Apparently, we can not simply increase the priority of the idle-GPOS task. Otherwise, the lower priority tasks will have no chance to execute.
- T_{SS2}. RG-RPC is returned from GPOS at this stage, so the RTOS kernel should resume τ_i as soon as possible. T_{SS2} can be calculated as

$$T_{SS2} = T_{rsc} + T_{rrc} + T_{sched} + T_{mrr} + T_{msg} + T_P^{'} \tag{3}$$

  Here T_{rsc} and T_{rrc} are same as the definitions in Eq. (2), while T_{mrr} and T_{msg} respectively link to the duration to restore the context of RTOS and the duration to save the context of GPOS by the monitor, and $T_P^{'}$ is the interference which is brought by other tasks. GPOS should be kept in the lowest priority while there is no RG-RPC requirement. If more than one RG-RPCs have been returned, the trigger task with the highest priority should be switched on firstly.
- T_G. T_G can be calculated as

$$T_G = T_{wait} + T_{gsc} + T_{grc} + T_{svc} + T_{other} \tag{4}$$

  T_{gsc} and T_{grc} respectively link to the duration to save and restore the context of tasks by GPOS. T_{svc} is the duration to serve the RPC request. T_{wait} is the time to wait for the other on-going operations to be finished. T_{other} is the duration that other GPOS components run before the control is transferred to RTOS.

The target of minimizing the T_{RG-RPC} can be equally converting to minimize the T_P, $T_P^{'}$, T_{wait}, and T_{other}. For minimizing T_P and $T_P^{'}$, the lowest number of preemption should take place during the RG-RPC, but the task with higher priority than τ_i should be able to pre-empt. Therefore, a new scheduling policy is needed to switch GPOS on/off in time other than the idle-scheduling strategy. For minimizing T_{wait} and T_{other}, we should consider three problems. Firstly, GPOS (normally is Linux which has no real-time scheduler) should switch on the RPC service code in the first time. Secondly, GPOS should efficiently inform RTOS to switch on when the RPC service has been finished. Thirdly, server side in GPOS should also support RPC priority. Moreover, as all know, page fault can affect time predictability seriously, so it should be avoided in the executing of RPC service. We have the following basic measures to cope with such problems.

1. We know that interrupts are responded by the processor and handled by the kernel [28], and as a comparison, other signals are responded by the kernel or more higher-level software. Therefore, interrupt is the most efficient method to minimize T_{wait}. Interrupts belong to GPOS have no influence on RTOS, and get CPU controlling at the first time when GPOS executing.
2. As similar as interrupts, software exception is a method for the low priority software to call (and immediately switch to) the services of high priority software. These methods are always enveloped as system calls or hypervisor calls, and can be used to inform the returning of RPC to RTOS and to give control to RTOS. Thus it will be a good selection to minimize T_{other}.
3. We know that priority channel systems [29] can give a natural class of channel systems, where messages carry a numeric priority, and higher-priority messages can supersede lower-priority messages by preceding them in the FIFO (First In First Out) communication buffers. Therefore, we can support message priority and caching messages during preemption depending on priority channel systems.
4. The kernel services and data structures may be configured to avoid being swapped out. Therefore, we can implement the RPC service as a kernel module.

Our real-time RG-RPC (RTRG-RPC) scheme is derived from the above philosophies.

5.2. Efficiency design of RTRG-RPC

This sub-section will describe the design of RTRG-RPC in TZDKS, so some TrustZone mechanisms are included in the description. However, the design can also be used in other consolidation systems.

5.2.1. Enhanced idle-scheduling policy

This policy is partly inspired by the priority inheritance protocol. Idle-scheduling guarantees the high utilization of CPU for GPOS, but does not provide the priority-based scheduling for the RPC requirement.

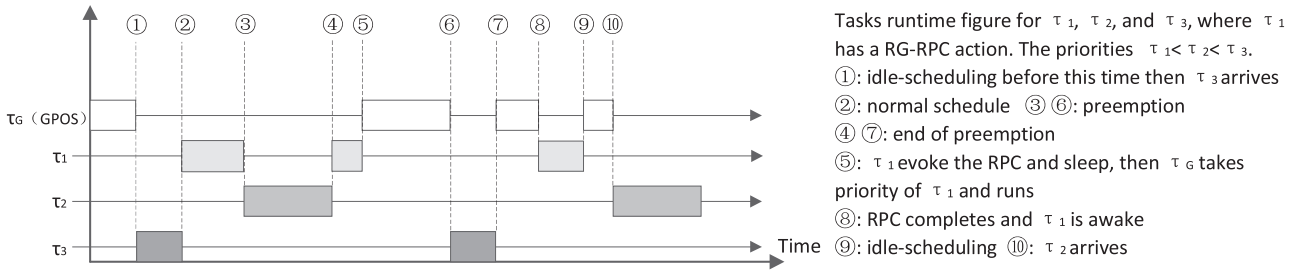


Fig. 4. Example of Scheduling for RTRG-RPC.

We need that GPOS has the same priority with the RPC caller (if there are more than one callers, GPOS has the highest priority of them) if needed, and has the lowest priority when there is no RPC requirement. Thus T_P and $T_P^{'}$ will be minimized without any violation of the FPS (Fixed Priority Scheduling) adopted in RTOS. We design an enhanced idle-scheduling mechanism for TZDKS, with the name - $Priority - Swapping$. It encompasses:

- Idle-scheduling plus τ_G. We hence add another real-time task τ_G also serving as a container of GPOS, but with a variable priority. τ_G owns a very low original priority in RTOS. When RPC service is needed, the priority of τ_G will be changed temporarily.
- Priority-Swapping. When a regular task τ_i of RTOS has triggered a RG-RPC call, it turns to sleep and exchanges its priority with τ_G. Then τ_G is scheduled with a higher priority originally belong to τ_i. When τ_G has done the service for the RPC, RTOS scheduler will reset priorities of both tasks to their original value, and resume τ_i.
- Timeout-exit strategy for τ_G. A timeout value can be setup for τ_i initially. If there is no RPC response yet after a certain duration, τ_G will be suspended for timeout. Meanwhile, its priority will be reset to the original value. τ_i is also restored to its old priority. This mechanism is proposed for the cause of the relatively lower reliability of the GPOS. Furthermore, if the frequency of RPC failure excess a threshold, the system will suspend τ_G, and broadcast the fault of τ_G to all other tasks.

We use an example (as shown in Fig 4) to explain the task scheduling in our TZDKS when a RTRG-RPC is triggered. Three RT tasks τ_1, τ_2, τ_3 run together with τ_G in the system. Their priority sequence is $\tau_1 < \tau_2 < \tau_3$. Before time ①, τ_1, τ_2, τ_3 are all free, so τ_G gets the opportunity to run. After that τ_3 arrives and executes immediately. τ_1 arrives in the period of τ_3 running but is switched on at time ② when τ_3 has finished. At the time ⑤, τ_1 triggers a RPC call and then goes to sleep. Meantime, the kernel swaps the priority of τ_1 and τ_G. Because at that time, τ_G is the ready task with the highest priority, it runs immediately. τ_G is preempted by arrived τ_2 at time ⑥ and resumed at time ⑦. At time ⑧, τ_G finishes the RPC service for τ_1, and that is detected by RTOS kernel, and the later restores the priority of both tasks, then awake τ_1.

5.2.2. RTRG-RPC Event path

In Eq. (4), T_{wait} contains the time that RPC service wait to run, and T_{other} contains the time to switch RTOS after the RPC service's completion. The latency is influenced by event path methods. The most efficient event approach is the interrupt. Considering that the RPC call will be sent to another OS but maybe on the same processor, we choose the software interrupt as the event method. The software interrupt is supported by almost all hardware systems now.

TrustZone specification recommends to use SMC as the communication method between GPOS and the secure world OS, and SMC is a kind of exception actually. This type of event method is not enough for RG-RPC, because normally only GPOS can request SMC service from the RTOS (in the secure world). The SGI (Software Generate Interrupt) is well-supported in the ARMv8 platform with GICv2 (or higher version) controller. Software in RTOS can generate SGI to inform GPOS of some events by writing GICD_SGIR register, which can be accessed by EL1 level (or higher priority level) instruct. Therefore, we adopt SGI as the event path from RTOS to GPOS (see Fig. 5), and SMC as the event path from GPOS to RTOS for the RTRG-RPC in TZDKS.

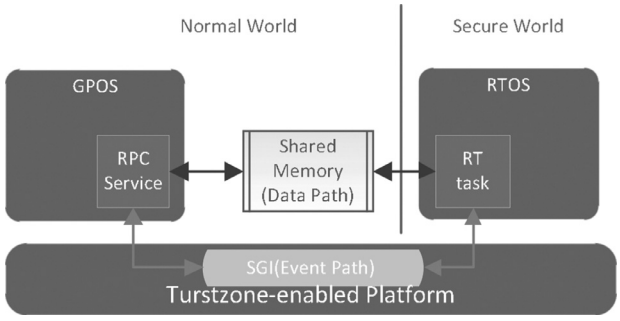


Fig. 5. Data Path and Event Path of RG-RPC.

On an ARM platform, a SGI signal to normal world will not really trigger an exception when the current CPU mode is secure, and the exception will take place after mode-switch. This hardware capability is pre-configured in GIC (Generic Interrupt Controler). When τ_i in RTOS calls kernel function to trigger a RG-RPC, the kernel will trigger a SGI signal and then implement priority-swapping for τ_i and τ_G. Thus when τ_G switches the GPOS on, the SGI interrupt handler will be invoked immediately. We envelop the RG-RPC service in the SGI handler function. So the time that RPC service waits to run can be minimized. At the end of RG-RPC service, a SMC call will be triggered by the service function. Then the system will switch to RTOS immediately. After that, the RTOS scheduler will awake the requester task of the RPC and try to switch it on. So the time to resume RTOS after the RPC service's completion can also be minimized.

5.2.3. RTRG-RPC data path

Both interrupts and exceptions can only carry data in the general registers, that is not enough for big amount of information transmitting, so we need an efficient data path in the RTRG-RPC scheme.

- Shared-Memory-based data channel
  RTRG-RPC uses the shared memory as the data channel (see Fig. 5), realized as two cache pools: a request pool and an answer pool.
- Channel Maintenance
  The first 64 bytes (e.g., the first slot) of each pool is defined as the pool-head, which is used for maintenance of the pool. Through the head, one can know or flag the occupation of the slot in the pool. Each byte of the head corresponds a RPC trigger's priority. The index of the head member is linked to the slot number in the pool, and the value of a head member remarks the priority of the RPC trigger task. If a message slot is free, the value of the corresponding $prio[i]$ is set to 255 (that is larger than the maximum value of the priority in RTOS). The value of $prio[0]$ is fixed to be 255, and the corresponding slot is unavailable. The consumer of the RPC message can easily know

whether there is a message for disposing or which slot is free in the pool by checking the head. After consuming the free slot or disposing a message, the software should modify the related member of the head.

- RG-RPC priority
  We design a simple method to setup the priority of RG-RPC. The consumer just finds the minimum *prio*[*i*] (the highest priority), and fetches linked message in the slot. Therefore, the producer of the RPC message need not do anything about priority.
- Slot size and efficiency
  Larger slot size does help for the increment of the bandwidth, but it increases the latency of RG-RPC yet. Note that the operation to read/write memory need considerable CPU cycles. So the slot size is the result of trade-off between bandwidth and latency. In TZDKS, Each pool possesses a 4k bytes page (can be customized to a larger page if needed in future), and contains 64 message slots. The 64-byte message length coincides with the cache line size of the ARMv8 CPU, which makes a high efficiency of the multi-level caches in the CPU. With two distinguishable pools in a RPC channel, it is helpful to decrease the occurrence of page faults, hence increases the speed of RPC message accessing.

5.2.4. RTRG-RPC service implementation in GPOS

To guarantee the response time and eliminate the uncertainty, we employ the interrupt handler in the GPOS kernel to serve RG-RPC call. That will avoid most indeterminacy factors such as page faults in memory accessing and scheduling delay. Considering the system efficiency affected by long hardirq critical region, we can place the RPC service into a high priority softirq (with the hypothesis that there is a limit number of hardirq per time unit).

Although GPOS is a non-real-time system designed for the maximum average performance (or maximum throughput), we are still able to make the service time determinable by increase the priority of the interrupt related to RG-RPC, and by simplifying the procedure of RPC service into a kernel module.

The GICv3 controller can support a minimum of 32 and a maximum of 256 levels of physical priority. All the interrupt types, including SPI, PPI, and SGI, can be set with a priority value. The priority of SGI is set by writing the register GICR_IPRIORITYR. GICv3 hardware guarantees that the unmasked interrupt with the highest priority will be firstly sent to the CPU core in bounded time. Now there is only one factor to delay the CPU interrupted by SGI signal, which is the interrupt masking. Interrupt masking duration is always the important optimizing object in the common GPOS, such as Linux. The scenarios to mask interrupt include system call handling, interrupt handling, and critical section in the kernel. Commonly their total number is limit, so we are able to get the maximum duration of all interrupt masking section, and can make the SGI interrupt to be handled in determinable time. Thus, it can be achieved that GPOS receives the RPC event in determinable time by defining the interrupt priority elaborately in TZDKS.

5.2.5. Scheduling feasibility verification in a RTRG-RPC supported RTOS

After adding Eqs. (2) - (7), the duration time of a complete RG-RPC without preemption can be counted as

$$\begin{aligned} T_{RG-RPC} &= T_I + 2T_{sched} + 2T_{rsc} + 2T_{rrc} + T_{mrg} \\ &\quad + T_{msg} + T_{mrr} + T_{gsc} + T_{svc} + T_R \end{aligned} \tag{5}$$

In Eq. (5), every part in the right hand is time predictable. Therefore, T_{RG-RPC} is time predictable.

We consider fixed-priority scheduling on an uni-processor system. A system consists of a task set T with n sporadic tasks. Each task is denoted as $\tau_i = (C_i, D_i, P_i, S_i)$ where C_i is the worst-case execution time (WCET), D_i is the relative deadline, and P_i is the minimum interarrival time between two successive releases of the task (sometimes also called period). S_i is the worst-case duration time (service time) of the RG-RPC without preemption, e.g., the worst-case value of Eq. (5). Here we need to count R_i, which is the worst-case response time for a task τ_i measured from the time the task τ_i is released. A task is said to have a constrained deadline if its relative deadline is smaller than or equal to its period.

To test the scheduling feasibility of the RTOS in TZDKS, We apply the method proposed in [30]. We can count the worst-case response time R_i by

$$R_i = C_i + S_i + \sum_{\forall j \in hp(i)} \left\lceil \frac{R_i}{P_i} \right\rceil (C_i + S_i) \tag{6}$$

It is possible to solve this equation using an iterative technique. Let R_i^n be the nth approximation to the true value of R_i. These approximations are generated from the above equation:

$$R_i^{n+1} = C_i + S_i + \sum_{\forall j \in hp(i)} \left\lceil \frac{R_i^n}{P_i} \right\rceil (C_i + S_i) \tag{7}$$

By use of mathematical induction, we can easily prove that $R_i^{n+1} \geq R_i^n$. So, we expect that the iteration starts with $R_i^0 = 0$ and terminates when $R_i^{n+1} = R_i^n$, while the iteration can be halted if $R_i^{n+1} \geq D_i$. Before the system deployment, we can compute each R_i off-line, and compare it to D_i to test the feasibility of real-time requirement.

5.3. Security design of RTRG-RPC

In the terminology of information system security, risks exists if there are latent vulnerability and threat. Vulnerability is the opportunity to cause damage. For the perspective of security, the communication path and the target system are viewed as an entirety. If any part exists an exploitable vulnerability, the communication will lead to threats. Unfortunately, the design of communication can not give absolute immunity from attacks. For example, the key step of the attack in [27] is to transfer some compliant data to trigger the buffer overflow in the secure OS, and it is almost impossible for the communication mechanisms to detect such behaviours. Conversely, the traditional transport layer security protocols use many heavyweight measures such as authentication, firewall, and intrusion detection, and these measures add the efficiency conflict in our target system. For the sake of feasibility, we only consider lightweight measures and limited threats. Based on these ideas, our security design mainly counters three types of threats.

The first is the safety threat. Our design should avoid or minimize the stability decrease of OSs in the consolidation system. The communication should have no side-effect for running, switch and restoring any OS. Because RTRG-RPC only provides a transport layer for communications, we only consider three cases that may disturb the related OSs: RPC no-return, RPC wrong return, and wrong order of multiple RPCs. RPC no-return means that GPOS has received the RPC request, but does not return the result to RTOS. In our enhanced idle-scheduling policy, we design the time-out exit mechanism, which will cope with this problem. RPC wrong return means that a returned message with wrong value or wrong format. The RPC wrong return can not be detected by this layer, so we leave it to be solved by the protocols or ways on the upper soft layer. The wrong order of multiple RPCs is caused by the concurrency of many RPCs, and it possibly bewilders the OS mechanism. The task ID attached in each RPC can help to solve this problem, and we limit that a task can not issue the next RPC request before the completion of the current task.

The second is the malicious code threat in the message, our design should prevent the executing of code in the buffer memory used by the communication. Our design only considers the data path for RTRG-RPC, because the event path can not carry any executed code. We leverage the DEP (Data Execution Protect) mechanism provided by hardware to forbid the code executing. DEP is an operating system feature that relies on processor hardware to mark memory with an attribute that indicates where code execution should not occur in the memory space. DEP prevents code execution from data pages in the default heap, memory stacks, and memory pools.

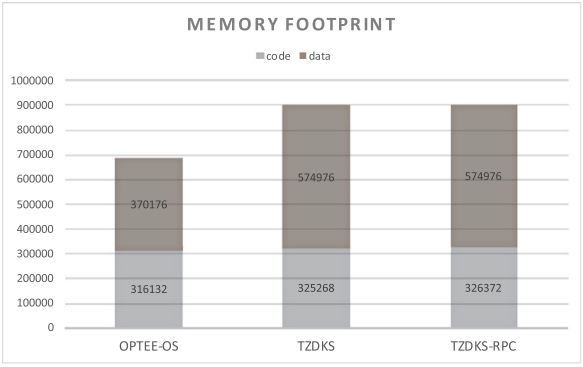


Fig. 6. Memory Footprint Comparison.

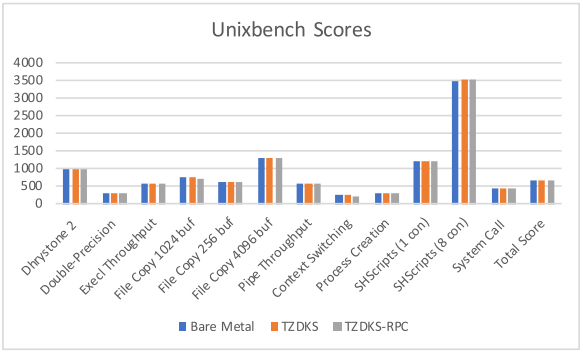


Fig. 7. UnixBench score for RTRG-RPC equipped TZDKS.

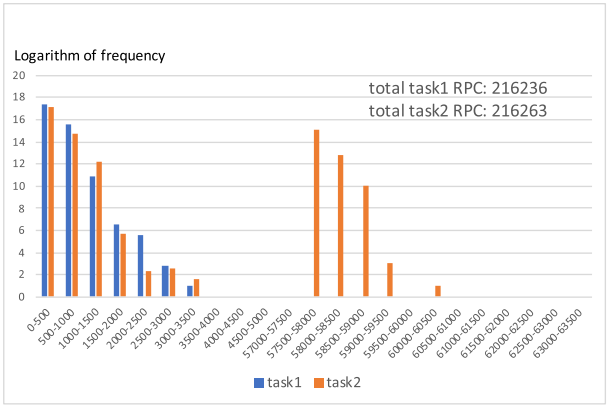


Fig. 8. Latency distribution of RTRG-RPC.

The third is the DoS attack threat. Any participant should not damage the availability of the other side OS. In TZDKS, RTOS is the key protected object because of its high priority. We set up a counter in the RPC service handler of RTOS to test the frequency of calling from GPOS. If the calling frequency exceeds a predefined threshold, RTOS will deprive some execution ticks from GPOS, so that the DoS threat can be eliminated.

Besides of these measures, we leave the other protects for confidentiality, integrity and so on to the upper-level security protocols in the OS.

6. Evaluation

We implemented TZDKS with RTRG-RPC supporting on a Hikey960 development board with TrustZone being enabled. The HiKey960 board is based around the Huawei Kirin 960 octa-core processor with four ARM Cortex-A73 (Max 2.4GHz) and four Cortex-A53 (Max 1.8GHz) cores with 4GB of SDRAM memory, 32GB of UFS 2.0 flash storage, platform peripherals (Watchdog timer, Real-time timer, and Power controller), and secure peripherals (Real-time clock, Trusted watchdog, Random number generator).

The RTOS is configured to execute on a Cortex-A53 core with the system timer at frequency 10,000Hz. Linux is driven by a 100Hz timer and can schedule all cores on the CPU. All the latency tests are measured by the reading of PMU (Performance Monitor Unit) registers in the ARM CPU core. We omit the service code in GPOS so that the service returns a SMC message immediately without any business function. Therefore, we mainly measure the detail of the state transitions for RTRG-RPC.

6.1. Software overhead

Firstly, we evaluate the memory footprint of RTRG-RPC and compare it with TZDKS, and compare it with the native OPTEE-OS (OPTEE is a popular and open-source TEE OS without scheduling capability). Fig. 6 presents the collected measurements. We can find that TZDKS only has 9KB codes more than that of OPTEE, and has 20KB data more than later. In order to implement RTRG-RPC, we only add about 1KB codes to TZDKS. Therefore, RTRG-RPC adds negligible footprint to the whole system.

Due to the lack of a method to test the integral performance of TZDKS, we use UnixBench to measure the comprehensive performance of Linux (GPOS) with zero load in RTOS. The results will reflect the performance of the whole system. When testing the performance influenced by RTRG-RPC, we configure the RT tasks trigger RPC with the frequency 15/s. As shown in Fig. 7, we can see that there is almost no performance loss in both GPOSs of TZDKS without/with RTRG-RPC. In all sub-tests, the performance metric mostly affected by RTRG-RPC is context-switch with the loss less than 4%. The total scores of results on three systems are respectively 673.2, 672.9, and 668.0. These results show that the overhead induced by RTRG-RPC is very little and acceptable.

6.2. Latency predictability and distribution

In this experiment, there are three periodic tasks τ_1, τ_2, τ_3 in RTOS, with completion time 10000, 1000, and 50,000 cycles, respectively, and a total load of RTOS is less than 69% so that RTOS is schedulable under FPS policy. τ_1 and τ_2 require RPC, and we give top priority to τ_1, so no preemption takes place in the process of τ_1 and its RPC. τ_2 owns the lowest priority of three tasks, so it and its RPC can be preempted by other tasks. We repeat RPC triggered by each task (τ_1, τ_2) for about 200 thousand times; meantime, we use UnixBench as a payload in GPOS to get a heavy resource usage rate and record the latency. Here we adjust the periods of τ_1 and τ_2 to make them triggers similar numbers of RPC in the same duration. Fig. 8 summarizes the statistic of latency for two tasks' RPC.

Because of the data concentration (i.e., more than 99.3% RPC calls of τ_1 return in 800 cycles), we use the logarithms of occurrence times as the scale of Y-axis. As shown in Fig 8, all RPC calls of τ_1 complete within 4000 cycles, and only 7 calls exceed 3000 cycles, which is the equivalent of 2μs on this CPU. This result shows that τ_1 and its RPC may not be preempted by other tasks and their RPC. When we focus on the latency statistic of τ_2's RPC, we can find that the data concentrates in two areas. One area is 0–3000, and the other is 50,000–60,000. This distribution indicates that τ_2's RPC may be preempted by the other tasks. However, if we get rid of the complete cycles of τ_1 and τ_3 from the larger latency values, the result will coincide with that of τ_1. The final result not only shows the predictability of RTRG-RPC but also verifies that the priority capability of RTRG-RPC.

In Table 3, we give the comparison of maximum (Max) latency, minimum (Min) latency, average latency, and the mean-square error (MSE)

Table 3
RTRG-RPC latency in different priorities.

	Max (cycles)	Min (cycles)	Average (cycles)	MSE (cycles)
Highest Priority	3867	103	311.8	283.8
Lower Priority	60,108	111	11906.4	23000.5

Table 4
Three RPC policies for evaluating.

Type	Description	Priority-swap in RTOS	Rapid service in GPOS
TRG-RPC	Traditional RPC	NO	NO
ITRG-RPC	Improved traditional RPC	NO	YES
RTRG-RPC	RT-RPC for TZDKS	YES	YES

Table 5
Latency comparison for three RG-RPCs.

	Max (cycles)	Min (cycles)	Average (cycles)	MSE (cycles)
RTRG-RPC	3267	69	302.9	275.6
ITRG-RPC	63,129	73	11909.1	22999.5
TRG-RPC	8,952,336	509	1759544.3	739267.5

of latencies for RTRG-RPC of τ_1 and τ_2. We can see that only the maximum RPC latency of τ_2 is significantly increased, which shows that RTRG-RPC scheme does not violate priority scheduling and is still predicable in a lower priority.

6.3. Latency comparison

In order to show the efficiency and the predictability of RTRG-RPC, we implement another two RPC methods in TZDKS. The first one is the traditional RPC method without any real-time consideration, and is labeled as TRG-RPC. TRG-RPC depends on the service process in GPOS, which runs under the standard idle-scheduling policy. The second policy is named ITRG-RPC, and it is the enhanced version of the first one, which uses the kernel interrupt handler as the RPC server in GPOS. We measured the response time of RG-RPC in three policies to evaluate the real-time performance of RTRG-RPC.

These three RPC policies are listed in Table 4. In fact, we combine the event path model and GPOS service model of RTRG-RPC into a bundled implementation, called *Rapid Service*. Therefore, the ITRG-RPC is the enhanced version of the traditional RPC with Rapid Service in GPOS, but the weaker version of the RTRG-RPC without priority-swap in RTOS. We believe that the details of the influence by each part improvement can be exposed by the comparative study of these policies.

The results of experiments for these policies are listed in Table 5. It can be learned that RTRG-RPC owns much higher efficiency and better predictability comparing to TRG-RPC and ITRG-RPC. We can also see that *Rapid Service* in GPOS do more significant help to RPC efficiency than priority-swap because ITRG-RPC owns comparative minimum latency than that of TRG-RPC. We note that the link in the GPOS is the most crucial link for the time efficiency of RTRG-RPC, and we will make further analysis in the next experiment.

6.4. Latency constituents

The previous test results show that the minimum latency of RTRG-RPC may be less than 100 cycles, which seems too short for a series of operations to complete. The ARM CPU provides many optimization measures to accelerate the executing of instruction, such as pipeLine, superscalar, out-of-order execution, etc. It's these optimizations that make the results seem so incredible. Thus, these measures may hide some cycles used in executing. In order to reveal the cost in every sub-section of RTRG-RPC detailedly, we run TZDKS on the Fast-Model platform, which is the simulator developed by the ARM official team. This platform does not provide various optimizations provided by physical processors.

Table 6
Latency of every link in RTRG-RPC.

	Min (cycles)	Max (cycles)	Ave (cycles)	MSE (cycles)
T_I	329	357	331.4	7.6
T_{rsc}	70	108	107.3	5.1
T_{rrc}	103	103	103	0
T_{mrg}	77	119	110.7	16.7
$T_{wait} + T_{gsc} + T_{svc}$	47	4361	115.9	617.7
T_{msg}	97	97	97	0
T_{mrr}	101	187	101.7	7.6
T_R	256	353	342.1	18.6
T_{sched}	107	107	107	0

Table 7
Comparison of computing value and experiment value for RTRG-RPC.

	Max (cycles)	Min (cycles)	Ave (cycles)
Experiment value	6867	2043	2078.4
Computing value	6875	2125	2431.6

Based on the Fast-Model, we do a more detailed test and statistic for the elements of Eq. (5). This may help to make further analysis and improvement to the RTRG-RPC. In this experiment, we can see that the minimum latency is about 2000 cycles, which is close to the real cycles consumed by a rapid RPC process.

From the results shown in Table 6, we can see that, besides the parts in GPOS, each other part in RTOS or monitor is very deterministic, and the maximum value is very close or equal to the minimum value. This result shows TZDKS can guarantee the real-time requirement in its RTOS, and also shows that the key to the efficiency lies in GPOS.

We find there are two main causes for the latency in GPOS. One is that some exception handler is executing when the TZDKS switches the GPOS on for this RTRG-RPC. In this condition, GPOS will disable its interrupt until the current exception completes. So the RPC service has to wait and generate more latency. The other cause is that an interrupt of real-time timer arrives before the completion of the RPC service in GPOS, so the TZDKS has to switch the RTOS on and another time of RTRG-RPC process will repeat when τ_G execute. In this condition, the latency of a RPC may multiply.

Some measures can be used to improve the RTRG-RPC scheme according to the above causes. Firstly, we can find out the exception handler with long latency, and try to limit the executing time of such handler. Secondly, we can limit the load of RTOS to decrease the occurrence of disturbing RPC service in GPOS by the timer.

We also use the results in Table 6 to count the value of $T_{RTRG-RPC}$ through equation (5). Then we compare the computing values with the experiment values, and list the result in Table 7. This comparison proposes verification to the correctness of both equation (5) and our process analysis for RTRG-RPC.

Table 8
Latency comparison with VMRPC and XMLRPC.

	RTRG-RPC TZDKS	VMRPC XEN	XMLRPC XEN
Lantency Cycles	$\approx 10^3$	$\approx 39 \times 10^3$	$\approx 718 \times 10^3$

6.5. Latency comparison with other RPC methods

We compare the efficiency of RTRG-RPC with the other existing inter-OS RPC protocols. We list the average latency of RTRG-RPC (with the top priority) and the results of the paper [31] in the Table 8. For the fairness of comparison, all results were normalized to the consuming cycles of the processor. The comparison shows that RTRG-RPC owns an order of magnitude advantage over existing inter-OS RPC methods.

7. Conclusions and future work

In this paper, we propose a Real-Time RG-RPC for a hybrid dual-OS system, and implement such a protocol in TZDKS. Time predictable control of RTRG-RPC is achieved through three mechanisms: SGI (Software Generate Interrupt) message transforming, interrupt handler RPC serving, and priority-swapping. The RTRG-RPC provides an ideal base for building efficient inter-operation in a hybrid multi-OS system.

While this paper verifies the feasibility of obtaining a real-time service from a GPOS, a number of issues of details within the GPOS have been ignored. There remains a number of obstacles in developing a practical RPC service in a hybrid dual-OS system. Our next plan is to address these issues and to extend the system model to a distributed and/or multi-core platform.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests:

We declare that we have no conflict of interest.

Acknowledgments

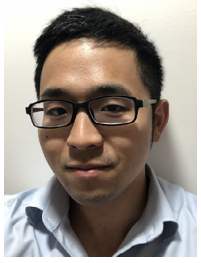
This work is supported by the National Natural Science Foundation of China (61602492, 61872444, 61502510). Much of the work reported in this paper took place while the first author was visiting the University of York.

References

[1] R. Berger, Consolidation in vehicle electronic architectures, Think: Aact, Jul (2015).
[2] M. Sato, G. Fukazawa, K. Yoshinaga, Y. Tsujita, A. Hori, M. Namiki, A hybrid operating system for a computing node with multi-core and many-core processors, Int. J. Adv. Comput. Science (IJACSci) 3 (7) (2013).
[3] S. Pinto, J. Pereira, T. Gomes, M. Ekpanyapong, A. Tavares, Towards a trustzone-assisted hypervisor for real-time embedded systems, IEEE Comput. Archit. Lett. 16 (2) (2016) 158–161.
[4] Q. Yu, H. Wei, M. Liu, T. Wang, A novel multi-os architecture for robot application, in: 2011 IEEE International Conference on Robotics and Biomimetics, IEEE, 2011, pp. 2301–2306.
[5] H. Wei, Z. Huang, Q. Yu, M. Liu, Y. Guan, J. Tan, Rgmp-ros: A real-time ros architecture of hybrid rtos and gpos on multi-core processor, in: 2014 IEEE International Conference on Robotics and Automation (ICRA), IEEE, 2014, pp. 2482–2487.
[6] Z. Jiang, N.C. Audsley, P. Dong, Bluevisor: A scalable real-time hardware hypervisor for many-core embedded systems, in: 2018 IEEE Real-Time and Embedded Technology and Applications Symposium (RTAS), IEEE, 2018, pp. 75–84.
[7] Z. Jiang, Real-Time I/O System for Many-core Embedded Systems, University of York, 2018 Ph.D. thesis.
[8] Q. Zhang, L. Liu, Shared memory optimization in virtualized cloud, in: Cloud Computing (CLOUD), 2015 IEEE 8th International Conference on, IEEE, 2015, pp. 261–268.
[9] C. Hao, P. Cuifen, S. Jianhua, S. Lin, Xenrpc: design and implementation of security vm remote procedure call, J. Comput. Res. Dev. 5 (2012).
[10] D. Sangorrín, S. Honda, H. Takada, Reliable and efficient dual-os communications for real-time embedded virtualization, Inf. Media Technol. 8 (1) (2013) 1–17.
[11] Z. Jiang, N. Audsley, P. Dong, N. Guan, X. Dai, L. Wei, Mcs-iov: Real-time i/o virtualization for mixed-criticality systems, in: 2019 IEEE Real-Time Systems Symposium (RTSS), IEEE, 2019, pp. 326–338.
[12] W. Liu, W. Ma, Y. Yang, Analysis and improvement of the ban modified andrew secure rpc protocol, J. Netw. 6 (4) (2011) 662.
[13] A. Vahidi, C. Jämthagen, Secure rpc in embedded systems: Evaluation of some globalplatform implementation alternatives, in: Proceedings of the Workshop on Embedded Systems Security, ACM, 2013, p. 4.
[14] X. Meng, K. Qian, D. Lo, H. Shahriar, M.A.I. Talukder, P. Bhattacharya, Secure mobile ipc software development with vulnerability detectors in android studio, in: 2018 IEEE 42nd Annual Computer Software and Applications Conference (COMPSAC), 1, IEEE, 2018, pp. 829–830.
[15] P. Dong, A. Burns, Z. Jiang, X. Liao, Tzdks: A new trustzone-based dual-criticality system with balanced performance, in: 2018 IEEE 24th International Conference on Embedded and Real-Time Computing Systems and Applications (RTCSA), IEEE, 2018, pp. 59–64.
[16] P.J. Prisaznuk, Arinc 653 role in integrated modular avionics (ima), in: Digital Avionics Systems Conference, 2008. DASC 2008. IEEE/AIAA 27th, IEEE, 2008, pp. 1–E.
[17] Z. Jiang, N. Audsley, P. Dong, Blueio: a scalable real-time hardware i/o virtualization system for many-core embedded systems, ACM Trans. Embedded Comput. Syst.(TECS) 18 (3) (2019) 1–25.
[18] Z. Jiang, N. Audsley, Vcdc: The virtualized complicated device controller, in: 29th Euromicro Conference on Real-Time Systems (ECRTS 2017), Schloss Dagstuhl-Leibniz-Zentrum fuer Informatik, 2017.
[19] J. Zhe, How to Build a Mixed-criticality System in Industry, 2019.
[20] S. Pinto, J. Pereira, T. Gomes, A. Tavares, J. Cabral, Ltzvisor: Trustzone is the key, in: LIPIcs-Leibniz International Proceedings in Informatics, 76, Schloss Dagstuhl-Leibniz-Zentrum fuer Informatik, 2017.
[21] W. Li, Y. Xia, H. Chen, B. Zang, H. Guan, Reducing world switches in virtualized environment with flexible cross-world calls, in: ACM SIGARCH Computer Architecture News, 43, ACM, 2015, pp. 375–387.
[22] L. Vilanova, M. Jordà, N. Navarro, Y. Etsion, M. Valero, Direct inter-process communication (dipc): Repurposing the codoms architecture to accelerate ipc, in: Proceedings of the Twelfth European Conference on Computer Systems, ACM, 2017, pp. 16–31.
[23] D. Du, Z. Hua, Y. Xia, B. Zang, H. Chen, Xpc: architectural support for secure and efficient cross process call, in: Proceedings of the 46th International Symposium on Computer Architecture, ACM, 2019, pp. 671–684.
[24] N. Santos, H. Raj, S. Saroiu, A. Wolman, Using arm trustzone to build a trusted language runtime for mobile applications, in: ACM SIGARCH Computer Architecture News, 42, ACM, 2014, pp. 67–80.
[25] F. Cerqueira, B. Brandenburg, A comparison of scheduling latency in linux, preempt-rt, and litmus rt, in: 9th Annual Workshop on Operating Systems Platforms for Embedded Real-Time Applications, SYSGO AG, 2013, pp. 19–29.
[26] D. Komaromy, Unbox Your Phone, Technical Report, taszksec, 2018.
[27] D. Shen, Exploiting trustzone on android, Black Hat USA (2015).
[28] I. Englander, A. Englander, The Architecture of Computer Hardware and Systems Software and Networking: An Information Technology Approach, Wiley, 2009.
[29] C. Haase, S. Schmitz, P. Schnoebelen, The power of priority channel systems, in: International Conference on Concurrency Theory, Springer, 2013, pp. 319–333.
[30] N. Audsley, A. Burns, M. Richardson, K. Tindell, A.J. Wellings, Applying new scheduling theory to static priority pre-emptive scheduling, Software Engineering Journal 8 (5) (1993) 284–292.
[31] H. Chen, L. Shi, J. Sun, Vmrpc: A high efficiency and light weight rpc system for virtual machines, in: 2010 IEEE 18th International Workshop on Quality of Service (IWQoS), IEEE, 2010, pp. 1–9.



Pan Dong is an associate professor in the school of computer at the National University of Defense Technology. His research interests include operating systems, real-time system, hardware assistant virtualization, and information security. He has a PhD in computer science from the National University of Defense Technology. He is a member of the ACM.



Zhe Jiang received his Ph.D. from University of York (2019). He is currently working as the staff engineer in Renesas Electronics and visit research associate in University of York.



Alan Burns is a Professor of the Department of Computer Science, University of York, U.K. He is a Fellow of the Royal Academy of Engineering, UK, and a Fellow of the IEEE. He has served as Chair of the IEEE Technical Committee on Real-Time Systems and has published widely (over 500 papers and articles, and 10 books).



Yan Ding received her Ph.D. degree in Computer Science from National University of Defense Technology (NUDT), China in 2014. She is now an Associate Professor in the School of Computer Science, NUDT. Her research area is system software, cloud computing and information security.



Jun Ma is an associate professor in the school of computer at the National University of Defense Technology. His research interests include operating systems, information security, trust computing and software engineering. Has a PhD in computer science from the National University of Defense Technology.